MeterMind — Model 3: LLM Prompting (Claude)

This notebook implements the **large pretrained LLM baseline** for MeterMind.

Why this is interesting

Claude has been trained on vast amounts of text including poetry, so it has implicit knowledge of iambic meter, Shakespearean register, and stress patterns. This is knowledge that neither the DP reorderer nor the GRU has access to. The question is whether that broad pretrained knowledge outperforms a task-specific trained model (GRU) or a rule-based symbolic system (DP).

Pipeline

```
test pairs → Claude API (prompted) → evaluate (MA, SP, G) → compare with Models 1 & 2
```

0. Install dependencies

```
%pip install anthropic pronouncing sentence-transformers transformers --quiet
```

1. Imports & configuration

```
import re, io, os, json, time, math
import pandas as pd
import matplotlib.pyplot as plt
import anthropic
from google.colab import files
from tqdm.notebook import tqdm

# ── Shared utilities ──────────────────────────────────────────────
print('Upload meter_utils.py')
files.upload()
from meter_utils import *

print('Upload eval_metrics.py')
files.upload()
from eval_metrics import *

# ── API ───────────────────────────────────────────────────────────
client = anthropic.Anthropic(api_key="sk-ant-api03-VZa1hDyn7gDuLM00b67WURK2s-Yp4SPjnTqUGgDhU6Y7C1PRRFLIfO6p1TzD98i3qPxorkAS
# ── Model ─────────────────────────────────────────────────────────
MODEL         = "claude-haiku-4-5-20251001"   # swap for claude-sonnet-4-6 for higher quality
MAX_TOKENS    = 100
RETRY_LIMIT   = 3
SLEEP_BETWEEN = 0.3

print('Config ready.')

```

```
Upload meter_utils.py
Choose Files No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving meter_utils.py to meter_utils (1).py
Upload eval_metrics.py
Choose Files No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
```

2. Load test set

Upload `training_pairs.csv`

```
import random
SEED = 42
random.seed(SEED)

uploaded = files.upload()
```

```
filename = list(uploaded.keys())[0]
df = pd.read_csv(io.StringIO(uploaded[filename].decode('utf-8')))
pairs = list(zip(df['input'].tolist(), df['target'].tolist()))
random.shuffle(pairs)

# Replicate exact same split as Model 2.
# Use sorted() to make unique_targets deterministic regardless of Python's
# set ordering — without this, the split changes between runs despite the seed.
unique_targets = sorted({t for _, t in pairs})
random.shuffle(unique_targets)
n = len(unique_targets)
test_targets = set(unique_targets[int(n * 0.9):])
test_pairs   = [(s, t) for s, t in pairs if t in test_targets]

print(f'Test pairs: {len(test_pairs):,}')
print(f'\nExample:')
print(f'  INPUT  : {test_pairs[0][0]}')
print(f'  TARGET : {test_pairs[0][1]}')

```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving training_pairs.csv to training_pairs (1).csv
Test pairs: 147

Example:
  INPUT  : Which did erst canopy the herd from heat, as a protective shelter doth.
```

3. Prompt design

The prompt needs to:

1. Tell Claude exactly what iambic pentameter is
2. Constrain it to only use words from the input
3. Preserve archaic register
4. Return only the rewritten line — no explanation

We use a **zero-shot** prompt aka no examples provided. This tests Claude's pretrained knowledge directly.

```
SYSTEM_PROMPT = """\
You are an expert in Shakespearean poetry and iambic pentameter.

Iambic pentameter is a line of 10 syllables with alternating unstressed and stressed beats:
  da-DUM da-DUM da-DUM da-DUM da-DUM

Your task: rewrite the given archaic prose line into iambic pentameter.

STRICT RULES:
1. Use ONLY words that appear in the input line — do not add new words.
2. You may reorder words freely.
3. Preserve the archaic register (thee, thou, thy, doth, hath, etc.).
4. Aim for exactly 10 syllables matching the da-DUM pattern.
5. Return ONLY the rewritten line — no explanation, no punctuation changes, no quotation marks.
"""

USER_TEMPLATE = "Rewrite this into iambic pentameter:\n{line}"


def prompt_claude(line):
    """Send a single line to Claude and return the rewritten version."""
    for attempt in range(RETRY_LIMIT):
        try:
            response = client.messages.create(
                model=MODEL,
                max_tokens=MAX_TOKENS,
                system=SYSTEM_PROMPT,
                messages=[{"role": "user", "content": USER_TEMPLATE.format(line=line)}]
            )
            return response.content[0].text.strip()
        except Exception as e:
            print(f'  Attempt {attempt+1} failed: {e}')
            time.sleep(2 ** attempt)
    return None

```

```
# Smoke test
test_out = prompt_claude(test_pairs[0][0])
print(f'INPUT  : {test_pairs[0][0]}')
print(f'TARGET : {test_pairs[0][1]}')
if test_out is None:
    raise RuntimeError("API call failed — check your ANTHROPIC_API_KEY before continuing.")
print(f'OUTPUT : {test_out}')
```

```
INPUT  : Which did erst canopy the herd from heat, as a protective shelter doth.
TARGET : Which erst from heat did canopy the herd,
OUTPUT : Which did erst canopy the herd from heat
```

4. Evaluation metrics

Same three metrics as Models 1 and 2.

```
# ── Load heavy models before the eval loop ──────────────────────────────
# meter_utils and eval_metrics were imported in Cell 4.
load_sp_model()
load_gpt2()
print('All metrics ready.')

```

```
Loading SentenceTransformer (first run downloads ~90 MB)...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
modules.json: 100%                                                  349/349 [00:00<00:00, 23.4kB/s]

config_sentence_transformers.json: 100%                                                  116/116 [00:00<00:00, 10.1kB/s]

README.md:        10.5k/? [00:00<00:00, 331kB/s]

sentence_bert_config.json: 100%                                                  53.0/53.0 [00:00<00:00, 1.26kB/s]

config.json: 100%                                                  612/612 [00:00<00:00, 52.7kB/s]

model.safetensors: 100%                                                  90.9M/90.9M [00:01<00:00, 210MB/s]

Loading weights: 100%                                                  103/103 [00:00<00:00, 453.57it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100%                                                  350/350 [00:00<00:00, 18.7kB/s]

vocab.txt:        232k/? [00:00<00:00, 9.56MB/s]

tokenizer.json:        466k/? [00:00<00:00, 15.2MB/s]

special_tokens_map.json: 100%                                                  112/112 [00:00<00:00, 9.33kB/s]

config.json: 100%                                                  190/190 [00:00<00:00, 18.4kB/s]
Loading GPT-2 (first run downloads ~500 MB)...
tokenizer_config.json: 100%                                                  26.0/26.0 [00:00<00:00, 2.10kB/s]

vocab.json:        1.04M/? [00:00<00:00, 6.03MB/s]

merges.txt:        456k/? [00:00<00:00, 3.11MB/s]

tokenizer.json:        1.36M/? [00:00<00:00, 4.49MB/s]

config.json: 100%                                                  665/665 [00:00<00:00, 59.4kB/s]

model.safetensors: 100%                                                  548M/548M [00:02<00:00, 279MB/s]

Loading weights: 100%                                                  148/148 [00:00<00:00, 490.86it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                  | Status     |  |
---------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
generation_config.json: 100%                                                  124/124 [00:00<00:00, 4.43kB/s]
All metrics ready.
```

4a. Sample run

Run the full eval pipeline on a small sample before committing to the whole test set to check the API key, metrics, and gives you a feel for output quality

```
# ── Sample eval: N_PREVIEW pairs through the full pipeline ──────────────────
N_PREVIEW = 5   # change to see more

_sample_results = []
_sample_failures = []

for _src, _tgt in tqdm(test_pairs[:N_PREVIEW], desc='Sample run'):
    _output = prompt_claude(_src)

    if _output is None:
        print(f"FAILED: {_src}")
```

```
        _sample_failures.append(_src)
        continue

    _words = tokenize(_output)
    _ma = metrical_accuracy(_words)
    _sp = semantic_preservation(' '.join(tokenize(_src)), _output)
    _g  = grammaticality(_output)

    # Word constraint check — warn if Claude added words not in input
    _violations = set(tokenize(_output)) - set(tokenize(_src))

    _sample_results.append({
        'input':      _src,
        'target':     _tgt,
        'output':     _output,
        'MA':         round(_ma, 3),
        'SP':         round(_sp, 3),
        'G':          round(_g,  3),
        'violations': sorted(_violations),
    })

    time.sleep(SLEEP_BETWEEN)

print(f"\n{len(_sample_results)} succeeded, {len(_sample_failures)} failed\n")
print(f"{'—'*70}")
for r in _sample_results:
    print(f"INPUT    : {r['input']}")
    print(f"TARGET   : {r['target']}")
    print(f"OUTPUT   : {r['output']}")
    print(f"MA={r['MA']:.3f}  SP={r['SP']:.3f}  G={r['G']:.3f}", end="")
    if r['violations']:
        print(f"  ⚠ added words: {r['violations']}", end="")
    print(f"\n")

```

```
Sample run: 100%                                                    5/5 [00:08<00:00,  1.53s/it]
`loss_type=None` was set in the config but it is unrecognized. Using the default loss: `ForCausalLMLoss`.

5 succeeded, 0 failed

——————————————————————————————————————————————————————————————————————
INPUT    : Which did erst canopy the herd from heat, as a protective shelter doth.
TARGET   : Which erst from heat did canopy the herd,
OUTPUT   : Which did erst canopy the herd from heat
MA=0.900  SP=0.889  G=0.092

INPUT    : And right perfection, which hath been wrongfully disgraced by villainous hands
TARGET   : And right perfection wrongfully disgraced,
OUTPUT   : And perfect right which hath been wrongfully by villainous hands disgraced
MA=0.700  SP=0.929  G=0.134  ⚠ added words: ['perfect']

INPUT    : The weary night doth claim my heavy eyelids, and to it shall they fall?
TARGET   : My heavy eyelids to the weary night?
OUTPUT   : The weary night doth claim my heavy eyes
MA=1.000  SP=0.893  G=0.120  ⚠ added words: ['eyes']

INPUT    : Time, spent in gentle numbers, passeth so idly hence, and is most foolishly consumed;
TARGET   : In gentle numbers time so idly spent;
OUTPUT   : Time spent in gentle numbers passeth hence
MA=1.000  SP=0.872  G=0.100

INPUT    : Despise not the wrinkles that come upon thee, for this thy golden time doth shine with splendour despite of such blem
TARGET   : Despite of wrinkles this thy golden time.
OUTPUT   : Despise not wrinkles that upon thee come, for this thy golden time doth shine with splendour
MA=1.000  SP=0.942  G=0.163
```

5. Run on test set

```
# —— Step 1 & 2: Claude API calls + Metrical Accuracy ————————————————————
rows     = []
failures = []

for src, tgt in tqdm(test_pairs, desc='Prompting Claude + MA'):
    output = prompt_claude(src)
    if output is None:
```

```
        failures.append(src)
        continue
    words = tokenize(output)
    rows.append({
        'input':  src,
        'target': tgt,
        'output': output,
        'MA':     round(metrical_accuracy(words), 4),
    })
    time.sleep(SLEEP_BETWEEN)

print(f'{len(rows):,} succeeded, {len(failures)} failed.')

```

```
Prompting Claude + MA: 100%                                    147/147 [02:56<00:00,  1.09s/it]
147 succeeded, 0 failed.
```

```
# ── Step 3: Semantic Preservation (batched encode) ──────────────────────────
from sentence_transformers import util

inputs_clean  = [' '.join(tokenize(r['input'])) for r in rows]
outputs_clean = [r['output'] for r in rows]

print(f'Encoding {len(rows):,} input+output pairs...')
embs = load_sp_model().encode(
    inputs_clean + outputs_clean,
    convert_to_tensor=True,
    show_progress_bar=True,
)

n = len(rows)
for i, r in enumerate(rows):
    r['SP'] = round(float(max(0.0, util.cos_sim(embs[i], embs[n + i]))), 4)

print('SP done.')

```

```
Encoding 147 input+output pairs...
Batches: 100%                                                  10/10 [00:04<00:00,  2.58it/s]
SP done.
```

```
# ── Step 4: Grammaticality (batched GPT-2, per-sequence loss) ──────────────
import torch, math
from tqdm.notebook import tqdm as tqdm_nb

BATCH_SIZE = 32
gpt2_model, gpt2_tok = load_gpt2()

if gpt2_tok.pad_token_id is None:
    gpt2_tok.pad_token_id = gpt2_tok.eos_token_id

outputs_list = [r['output'] for r in rows]
g_scores     = []

for batch_start in tqdm_nb(range(0, len(outputs_list), BATCH_SIZE), desc='Grammaticality'):
    batch = outputs_list[batch_start : batch_start + BATCH_SIZE]
    enc   = gpt2_tok(batch, return_tensors='pt', padding=True,
                     truncation=True, max_length=128)
    input_ids = enc['input_ids']
    labels    = input_ids.clone()
    labels[input_ids == gpt2_tok.pad_token_id] = -100

    with torch.no_grad():
        logits = gpt2_model(**enc).logits

    shift_logits = logits[:, :-1, :].contiguous()
    shift_labels = labels[:, 1:].contiguous()
    loss_fn      = torch.nn.CrossEntropyLoss(ignore_index=-100, reduction='none')
    token_losses = loss_fn(
        shift_logits.view(-1, shift_logits.size(-1)),
        shift_labels.view(-1),
    ).view(len(batch), -1)
    mask       = (shift_labels != -100).float()
    seq_losses = (token_losses * mask).sum(-1) / mask.sum(-1).clamp(min=1)

```

```
    for loss_val in seq_losses.tolist():
        g_scores.append(round(1 / (1 + math.log(math.exp(loss_val))), 4))

for r, g in zip(rows, g_scores):
    r['G'] = g

print(f'G done. {len(g_scores):,} scores computed.')

```

Grammaticality: 100% 5/5 [00:54<00:00, 10.99s/it]

```
G done. 147 scores computed.
```

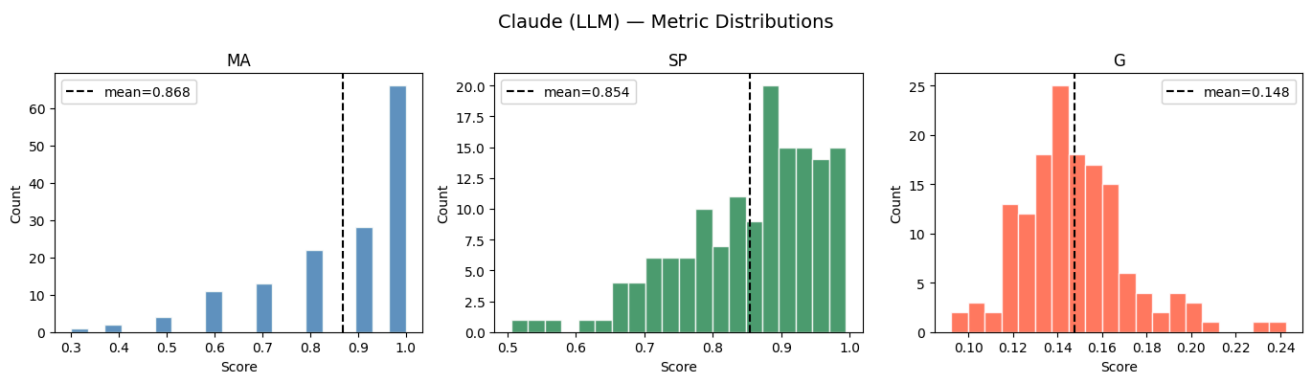
```
# ── Step 5: Assemble results DataFrame ──────────────────────────────────────
results_df = pd.DataFrame(rows)
print(f'Done. {len(results_df):,} results, {len(failures)} failures.')

```

```
Done. 147 results, 0 failures.
```

6. Results

```
print('=== Claude (LLM) — Test Set Results ===')
print(f"Metrical Accuracy (MA)    : {results_df['MA'].mean():.4f} ± {results_df['MA'].std():.4f}")
print(f"Semantic Preservation (SP): {results_df['SP'].mean():.4f} ± {results_df['SP'].std():.4f}")
print(f"Grammaticality (G)        : {results_df['G'].mean():.4f} ± {results_df['G'].std():.4f}")

print('\n=== Sample outputs ===')
for _, row in results_df.head(5).iterrows():
    print(f"INPUT  : {row['input']}")
    print(f"TARGET : {row['target']}")
    print(f"OUTPUT : {row['output']}")
    print(f"MA={row['MA']:.3f}  SP={row['SP']:.3f}  G={row['G']:.3f}")
    print()

# Distribution plots
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
fig.suptitle('Claude (LLM) — Metric Distributions', fontsize=14)
for ax, metric, colour in zip(axes, ['MA', 'SP', 'G'], ['steelblue', 'seagreen', 'tomato']):
    ax.hist(results_df[metric], bins=20, color=colour, edgecolor='white', alpha=0.85)
    ax.axvline(results_df[metric].mean(), color='black', linestyle='--', linewidth=1.5,
               label=f"mean={results_df[metric].mean():.3f}")
    ax.set_title(metric)
    ax.set_xlabel('Score')
    ax.set_ylabel('Count')
    ax.legend()
plt.tight_layout()
plt.savefig('llm_metrics.png', dpi=150)
plt.show()
```

```
=== Claude (LLM) — Test Set Results ===
Metrical Accuracy (MA)    : 0.8680 ± 0.1600
Semantic Preservation (SP): 0.8540 ± 0.1020
Grammaticality (G)        : 0.1475 ± 0.0246

=== Sample outputs ===
INPUT  : Which did erst canopy the herd from heat, as a protective shelter doth.
TARGET : Which erst from heat did canopy the herd,
OUTPUT : Which did erst canopy the herd from heat
MA=0.900  SP=0.889  G=0.092

INPUT  : And right perfection, which hath been wrongfully disgraced by villainous hands
TARGET : And right perfection wrongfully disgraced,
OUTPUT : And right perfection hath been wrongfully by villainous hands disgraced
MA=0.900  SP=0.964  G=0.121

INPUT  : The weary night doth claim my heavy eyelids, and to it shall they fall?
TARGET : My heavy eyelids to the weary night?
OUTPUT : The weary night doth claim my heavy eyes
MA=1.000  SP=0.893  G=0.119

INPUT  : Time, spent in gentle numbers, passeth so idly hence, and is most foolishly consumed;
TARGET : In gentle numbers time so idly spent;
OUTPUT : Time spent in gentle numbers passeth hence
MA=1.000  SP=0.872  G=0.100

INPUT  : Despise not the wrinkles that come upon thee, for this thy golden time doth shine with splendour despite of such blemis
TARGET : Despite of wrinkles this thy golden time.
OUTPUT : Despise not wrinkles that upon thee come
MA=1.000  SP=0.708  G=0.126
```



7. Save results

```
results_df.to_csv('llm_results.csv', index=False)
print('Saved llm_results.csv')

summary = {
    'model': 'Claude (LLM)',
    'n':     len(results_df),
    'MA':    {'mean': round(float(results_df['MA'].mean()), 4), 'std': round(float(results_df['MA'].std()), 4)},
    'SP':    {'mean': round(float(results_df['SP'].mean()), 4), 'std': round(float(results_df['SP'].std()), 4)},
    'G':     {'mean': round(float(results_df['G'].mean()),  4), 'std': round(float(results_df['G'].std()),  4)},
}
with open('llm_summary.json', 'w') as f:
    json.dump(summary, f, indent=2)
print('Saved llm_summary.json')

files.download('llm_results.csv')
files.download('llm_summary.json')
files.download('llm_metrics.png')
```

```
Saved llm_results.csv
Saved llm_summary.json
```

Mini Eval — Cross-Model Sample Comparison

Runs the full eval pipeline on a small fixed sample from the test set. Use the same `N_EVAL` and `SEED` across all three notebooks to ensure the comparison is fair.

```
# ── Mini eval ──────────────────────────────────────────────────────────
import random
from tqdm.notebook import tqdm

N_EVAL = 50
SEED   = 42

random.seed(SEED)
_eval_pairs = random.sample(test_pairs, min(N_EVAL, len(test_pairs)))

# ── Step 1: Claude API calls + MA ──────────────────────────────────────
_rows = []
for _src, _tgt in tqdm(_eval_pairs, desc='Claude + MA'):
    _output = prompt_claude(_src)
    if _output is None:
        continue
    _words = tokenize(_output)
    _rows.append({
        'input':  _src,
        'target': _tgt,
        'output': _output,
        'MA':     round(metrical_accuracy(_words), 4),
    })
    time.sleep(SLEEP_BETWEEN)

# ── Step 2: Semantic Preservation (batched) ────────────────────────────
from sentence_transformers import util as _util

_inputs_clean  = [' '.join(tokenize(r['input'])) for r in _rows]
_outputs_clean = [r['output'] for r in _rows]
_embs = load_sp_model().encode(
    _inputs_clean + _outputs_clean,
    convert_to_tensor=True,
    show_progress_bar=True,
)
_n = len(_rows)
for _i, _r in enumerate(_rows):
    _r['SP'] = round(float(max(0.0, _util.cos_sim(_embs[_i], _embs[_n + _i]))), 4)

# ── Step 3: Grammaticality (batched GPT-2) ─────────────────────────────
import torch as _torch, math as _math

_G_BATCH  = 32
_gm, _gt  = load_gpt2()
if _gt.pad_token_id is None:
    _gt.pad_token_id = _gt.eos_token_id

_out_list = [r['output'] for r in _rows]
_g_scores = []
for _bs in range(0, len(_out_list), _G_BATCH):
```